Semana 34 — Conteúdo de 2 aulas

Banco de Dados

Ricardo Pasquati Pontarolli

Técnico em Redes de Computadores · IFSP Boituva · pasquati@ifsp.edu.br · v9

Semana 34 — Conteúdo de 2 aulas

- **Aula 55:** INNER JOIN na Prática
- **Aula 56:** LEFT e RIGHT JOIN

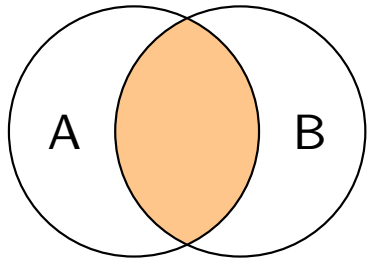
Aula 55 — INNER JOIN na Prática

Objetivos da Aula

- Dominar o **INNER JOIN**.
- Juntar **várias** tabelas.
- Combinar JOIN com WHERE e GROUP BY.

INNER JOIN

- Retorna apenas as linhas que **casam** nos dois lados.
- É o JOIN **mais comum**.
- `JOIN` sozinho já significa **INNER**.



INNER = apenas a interseção

Exemplo

```sql
SELECT c.nome, p.data, p.valor
FROM Cliente c
INNER JOIN Pedido p ON p.id_cliente = c.id;
```

- Só clientes **com** pedidos aparecem.

Juntando Três Tabelas

```
SELECT c.nome, pr.nome, i.quantidade
FROM Cliente c
JOIN Pedido p ON p.id_cliente = c.id
JOIN ItemPedido i ON i.id_pedido = p.id
JOIN Produto pr ON pr.id = i.id_produto;
```

- Cada JOIN **encadeia** uma tabela.

JOIN + WHERE

```sql
SELECT c.nome, p.valor
FROM Cliente c
JOIN Pedido p ON p.id_cliente = c.id
WHERE p.valor > 100;
```

JOIN + GROUP BY

```sql
SELECT c.nome, SUM(p.valor) AS total
FROM Cliente c
JOIN Pedido p ON p.id_cliente = c.id
GROUP BY c.nome;
```

- Total gasto por **cliente**.

> **Note**
>
> O JOIN **filtra implicitamente**: quem não tem correspondência fica de fora — por isso o INNER é o mais restritivo.

Prática

- Junte cliente e pedido.
- Junte três tabelas.
- Combine com GROUP BY.

Resumo da Aula

- **INNER JOIN** mantém só o que **casa**.
- Encadeia **várias** tabelas.
- Combina com WHERE e GROUP BY.

Referências

- ROB, P.; CORONEL, C. *Sistemas de Banco de Dados*. Cengage.

Aula 56 — LEFT e RIGHT JOIN

Objetivos da Aula

- Compreender os **outer joins**.
- Usar **LEFT** e **RIGHT JOIN**.
- Identificar registros **sem** correspondência.

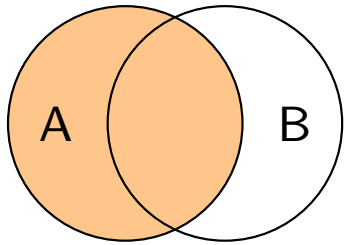
O Limite do INNER

- O INNER **descarta** linhas sem par.
- E se quisermos ver clientes **sem** pedidos?
- Precisamos de outro tipo de JOIN.

LEFT JOIN

```sql
SELECT c.nome, p.valor
FROM Cliente c
LEFT JOIN Pedido p ON p.id_cliente = c.id;
```

- **Todos** os clientes aparecem.
- Sem pedido → colunas do pedido ficam **NULL**.

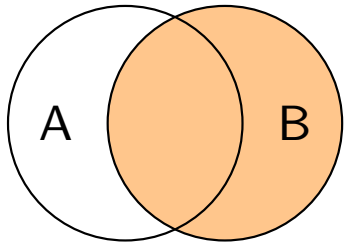


LEFT: tudo de A

RIGHT JOIN

```sql
SELECT c.nome, p.valor
FROM Cliente c
RIGHT JOIN Pedido p ON p.id_cliente = c.id;
```

- **Todos** os pedidos aparecem.
- Mesmo sem cliente correspondente.



RIGHT: tudo de B

O que é NULL aqui

- Quando não há par, as colunas do outro lado vêm **nulas**.
- Serve para **detectar** falta de correspondência.

Encontrar os “sem par”

```sql
SELECT c.nome
FROM Cliente c
LEFT JOIN Pedido p ON p.id_cliente = c.id
WHERE p.id IS NULL;
```

- Clientes que **nunca** compraram.

> **Note**
>
> O padrão `LEFT JOIN ... WHERE ... IS NULL` é a receita clássica para achar “quem **não** tem” correspondência.

Prática

- Liste todos os clientes e seus pedidos.
- Ache clientes **sem** pedidos.

Resumo da Aula

- **LEFT/RIGHT JOIN** mantêm um dos lados **completo**.
- Sem par $\rightarrow$ **NULL**.
- Úteis para achar **registros órfãos**.

Referências

- ROB, P.; CORONEL, C. *Sistemas de Banco de Dados*. Cengage.